SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Code of Conduct:

*I **SANTHOSH R (192321007)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

SANTHOSH R

Signature of the student:

Question 1: Online Banking: Fund Transfer Boundary Failures

Scenario: Fund transfers fail at minimum and maximum limits; the team relies on random testing.

1. Analysis of the current strategy

- Failures cluster at the boundaries, which points to off-by-one defects (for example > coded where >= was required). Such defects hide exactly at the edges of a range.
- Random testing draws values mostly from the middle of a wide range. If the valid range is ₹100 to ₹1,00,000, the chance of randomly hitting exactly ₹100 or ₹1,00,000 is roughly 1 in 1,00,000 per draw.
- There is no evidence of systematic partitions (invalid, zero, negative, non-numeric, over-balance, over-daily-limit) or of a defined expected result for each test.
- Assumed limits: minimum ₹100, maximum ₹1,00,000 per transfer.

2. Equivalence Class Partitioning (ECP)

ID	Input condition	Type	Representative value	Expected result
EC1	Amount < 100 (including 0 and negatives)	Invalid	50	Rejected: below minimum
EC2	100 ≤ Amount ≤ 1,00,000	Valid	25,000	Transfer successful
EC3	Amount > 1,00,000	Invalid	1,50,000	Rejected: above limit
EC4	Non-numeric or blank	Invalid	abc / (empty)	Validation error

ID	Input condition	Type	Representative value	Expected result
EC5	More than 2 decimal places	Invalid	500.555	Validation error
EC6	Amount in range but > available balance	Invalid	20,000 with balance 10,000	Rejected: insufficient funds

3. Boundary Value Analysis (BVA, including robustness values)

Test	Amount (₹)	Position	Expected result
B1	99.99	Just below minimum	Reject
B2	100.00	Minimum	Accept
B3	100.01	Just above minimum	Accept
B4	99,999.99	Just below maximum	Accept
B5	1,00,000.00	Maximum	Accept
B6	1,00,000.01	Just above maximum	Reject
B7	0 and -1	Robustness (invalid)	Reject, no balance change
B8	Amount = balance / balance − 0.01	Balance boundary	Accept (balance becomes 0) / Reject

4. Is random testing worth keeping?

Verdict: retain it only as a supplement; replace it as the primary technique. Random (or fuzz/property-based) testing is cheap to automate and can reveal unexpected inputs that no one designed a test for. However, it has no guarantee of covering boundaries, produces many redundant tests, and needs an oracle to be useful. Systematic ECP + BVA hits every boundary deterministically with about 14 tests, a number that random testing cannot match even with thousands of runs.

5. Proposed optimized approach

- Step 1: Extract limits and rules from the requirement (min, max, daily limit, balance rule).
- Step 2: Partition inputs with ECP, then derive boundary tests with BVA, including invalid boundaries.
- Step 3: Use a small decision table for combinations of amount, balance and daily limit.
- Step 4: Automate the suite in the regression pack; run it on every build.
- Step 5: Keep seeded random/fuzz runs (with a reproducible seed and a clear oracle) as a secondary exploratory layer.
- Exit criteria: all equivalence classes covered, all boundaries verified, zero open boundary defects.

6. Justification

Defects concentrate at boundaries, so boundary-driven design gives the highest defect-detection rate per test. The approach is repeatable, traceable to requirements, and measurable (class coverage and boundary coverage), which random testing alone is not.

Question 2: Healthcare Module: Post-Deployment Defects from Poor Coverage

Scenario: A healthcare module was tested but critical defects surfaced after deployment because of insufficient coverage.

1. Evaluation of the testing approach and gaps in adequacy criteria

- The suite appears to have been designed to execute statements, not decisions. Statement coverage can reach 100% while whole branches stay untested.

- No explicit test adequacy criterion (branch, path, condition) was defined as an exit criterion, so 'enough testing' was a judgement call.
- Defects usually sit in combinations (for example overdose in a child patient), which need path-level thinking.

2. Conceptual module and Control Flow Graph (CFG)

Conceptual prescription-validation module (assumed):

Node	Statement / decision
N1	Read age, weight, dose, allergyFlag
N2	Decision: if (age ≤ 0 OR weight ≤ 0)
N3	Return ERROR
N4	Decision: if (allergyFlag)
N5	Reject prescription
N6	Decision: if (dose > maxDose(weight))
N7	Reduce dose and raise alert
N8	Decision: if (age < 12)
N9	Pediatric safety check
N10	Approve prescription

Edges: N1→N2; N2→N3 (T); N2→N4 (F); N4→N5 (T); N4→N6 (F); N6→N7 (T); N6→N8 (F); N7→N8; N8→N9 (T); N8→N10 (F); N9→N10; plus N3, N5, N10 → Exit.

Cyclomatic complexity: V(G) = E − N + 2 = 14 − 11 + 2 = **5** (equivalently 4 decision nodes + 1). Feasible distinct paths = 6.

3. Coverage achieved versus coverage needed

Assume the original suite had three tests: T1 invalid input (N3), T2 allergy (N5), T3 overdose in a child (N1-N2-N4-N6-N7-N8-N9-N10).

Criterion	Achieved by T1-T3	Result
Statement coverage	N3, N5, N7, N9, N10 all executed	100% (looks complete, but misleading)
Branch coverage	N6-false and N8-false never taken	6 of 8 branches = 75%
Path coverage	3 of 6 feasible paths	50%

Missing paths: (a) normal dose, adult (N6 F, N8 F); (b) normal dose, child (N6 F, N8 T); (c) overdose, adult (N6 T, N8 F). These are exactly the real-world combinations that fail in production.

4. Improved testing strategy

Test	Inputs (example)	Path covered	Expected
P1	age 0, weight 60	N1-N2-N3	ERROR
P2	age 30, allergy = true	N1-N2-N4-N5	Rejected
P3	age 30, dose normal	N1-N2-N4-N6-N8-N10	Approved
P4	age 30, dose > max	N1-N2-N4-N6-N7-N8-N10	Reduced + alert
P5	age 8, dose normal	N1-N2-N4-N6-N8-N9-N10	Pediatric check, approved
P6	age 8, dose > max	N1-N2-N4-N6-N7-N8-N9-N10	Reduced + pediatric check

Test	Inputs (example)	Path covered	Expected
P7	age -1 with weight 60; age 30 with weight 0	N2 condition-level (MC/DC)	ERROR for each
P8	age 11, 12; dose = max, max+1	BVA on age and dose	Correct branch each time

5. Justification and impact on reliability

- P1-P6 give 100% statement, 100% branch and 100% path coverage for this module (all 6 feasible paths).
- P7 and P8 add condition-level and boundary rigor for the compound decision at N2 and the thresholds at N6 and N8.
- Setting branch coverage ≥ 100% (and path coverage for critical modules) as an exit criterion removes the guesswork about when testing is enough.
- In a healthcare setting each untested path is a potential patient-safety risk, so closing them lowers the probability of critical field defects.

Question 3: E-Commerce: Missed Errors in Input Combinations

Scenario: Price, quantity and discount fields interact; testers missed errors in specific combinations.

1. Limitations of the current black-box approach

- Fields appear to have been tested one at a time, so interaction faults (for example a discount larger than price × quantity) are invisible.
- Testing only valid values leaves invalid and boundary classes uncovered.
- Exhaustive combination is impractical: even 7 values per field gives $7^3 = 343$ tests, so a selection technique is needed.

Assumed rules: price ₹1 to ₹10,000; quantity 1 to 100; discount 0% to 50%; bulk discount applies only if quantity ≥ 10.

2. ECP and BVA per field

Field	Valid class	Invalid classes	Boundary values
Price	1 to 10,000	≤ 0; > 10,000; non-numeric	0, 1, 2, 9,999, 10,000, 10,001
Quantity	1 to 100	≤ 0; > 100; decimal; non-numeric	0, 1, 2, 99, 100, 101
Discount %	0 to 50	< 0; > 50; non-numeric	-1, 0, 1, 49, 50, 51

3. Decision table for combinations

Rule	Qty ≥ 10	Discount entered	Discount ≤ 50%	Inputs valid	Action
R1	N	N	-	Y	Total = price × qty
R2	N	Y	Y	Y	Reject: discount needs bulk quantity
R3	Y	N	-	Y	Total = price × qty
R4	Y	Y	Y	Y	Total = price × qty × (1 − d)
R5	Y	Y	N	Y	Reject: discount above cap
R6	-	-	-	N	Reject: validation error

4. Revised test design strategy

- Single-fault robustness BVA: 6n + 1 = 19 tests (n = 3 fields) covers every boundary of every field.
- One test per decision-table rule (6 tests) covers the logical combinations.
- Pairwise (all-pairs) set over the key values of price, quantity and discount: about 49 tests instead of 343, catching essentially all two-factor interaction faults.
- Targeted cases: discount at its cap with quantity exactly 10 and 9; maximum price × maximum quantity (overflow/rounding); price with 2 decimals × discount (rounding).
- Computed totals are checked against an independent oracle (spreadsheet formula).

5. Justification

ECP/BVA remove redundancy within a field, the decision table captures the business logic between fields, and pairwise combination covers interactions at a fraction of the exhaustive cost. Together they raise defect detection and give measurable completeness (all classes, all boundaries, all rules, all pairs).

Question 4: Nested Conditions and Loops: Partial Path Coverage

Scenario: A module with nested conditions and loops has only partial path coverage.

1. Evaluation and gaps

- The team likely drew a CFG with merged or missing nodes (loop back-edges and compound conditions often get simplified away), so some independent paths were never identified.
- Loops were probably tested only in the 'typical' case. Zero-iteration and boundary-iteration cases are where nested-loop defects live.
- Impact: untested independent paths are exactly where unexpected behavior hides, so defect detection drops and the coverage figure gives false confidence.

2. Representative module and CFG

Node	Code
1	i = 0
2	while (i < n) [P1]
3	if (rec[i].valid) [P2]
4	j = 0
5	while (j < m) [P3]
6	if (rec[i].f[j] == key) [P4]
7	count++
8	j++ (then back to 5)
9	i++ (then back to 2)
10	return count

Predicate nodes P1 to P4, so V(G) = 4 + 1 = **5** independent paths.

3. Independent (basis) paths

Path	Condition outcomes	Test data idea
IP1	P1 false immediately	n = 0 (empty set)
IP2	P1 T, P2 F, then P1 F	n = 1, record invalid (skipped)
IP3	P1 T, P2 T, P3 F	n = 1, valid record, m = 0 (inner loop skipped)
IP4	P1 T, P2 T, P3 T, P4 F	valid record, m ≥ 1, no match
IP5	P1 T, P2 T, P3 T, P4 T	valid record, m ≥ 1, match found

4. Complete path testing strategy (adds loop testing)

Loop	Cases to test
Outer loop (n)	0, 1, 2, typical, max − 1, max, max + 1 iterations
Inner loop (m)	0, 1, 2, typical, max − 1, max, max + 1 iterations (test innermost first, hold the outer at minimum)
Nested interplay	Outer at typical while inner at min, then inner at max; mixed valid/invalid records in one run

After the five basis paths pass, add combination paths that matter to the business logic (for example a valid record followed by an invalid one, then a match).

5. Justification with coverage criteria and expected outcomes

- Independent-path coverage (5 of 5) guarantees every statement and every branch executes at least once, since the basis set covers all edges.
- Loop-boundary tests expose off-by-one, uninitialised counter and termination errors that path coverage alone does not.
- Expected outcome: 100% statement, 100% branch, 100% basis-path coverage, plus all loop boundaries, with each test having a defined expected result for count.

Question 5: Financial Application: Hybrid Strategy Still Leaks Defects

Scenario: Black-box and white-box testing are both used, but defects keep appearing in production.

1. Critical analysis of the current integration

- **Siloed techniques:** black-box and white-box suites were probably designed independently, so they overlap on easy cases and both miss complex combinations.
- **Weak adequacy criteria:** coverage was likely reported as a single statement-coverage percentage. That does not show branch, condition or boundary adequacy.
- **Metrics misapplied:** a high coverage number was treated as a quality guarantee. Coverage only measures what was executed, not whether the oracle checked the right result.
- **No feedback loop:** production defects were not fed back to find which class or path was missing.

2. Proposed hybrid strategy

Layer	Technique	Target / adequacy criterion
Requirements-based (black box)	ECP on every input; BVA with invalid boundaries (amounts, rates, dates, tenure)	100% classes, 100% boundaries
Combinations (black box)	Decision tables for eligibility, fee and interest rules	One test per rule
Structure-based (white box)	CFG for each module; basis-path testing	100% statement, 100% branch, all V(G) independent paths
Critical decisions	MC/DC on compound conditions (approval, limit checks)	Each condition shown to affect the outcome
Loops	Zero / one / many / max / max+1 iteration tests	All loop boundaries
Gap closure	Run black-box suite, measure coverage, add white-box tests for uncovered paths	No unexplained uncovered branch

3. Workflow

- Design black-box tests from requirements first (they are independent of the code).
- Run them with a coverage tool; analyse uncovered branches and paths.
- Add path and MC/DC tests only for the gaps; remove duplicate tests.
- Review each production defect: classify it (missing class, boundary, path, oracle) and add a regression test for it.
- Gate release on defined exit criteria, not a single percentage.

4. Justification

Black-box testing checks that the software does what the requirement says (including code that was never written), while white-box testing checks that all of the code that was written has been exercised. Using one to fill the other's gaps, under explicit adequacy criteria, increases coverage and lowers the number of escaped defects. The feedback loop turns each escaped defect into permanent protection.

Conclusion: A requirement-driven black-box core, completed by structural coverage targets and a defect-feedback loop, gives the strongest defect-prevention for a financial application.

Question 6: HR Payroll: Bonus Errors at 5, 10 and 15 Years

Scenario: Employees at exactly 5, 10 and 15 years of service received incorrect bonus payouts.

1. Failure analysis

The errors occur exactly on the band edges, so the defect is an off-by-one in the comparison operator (for example years > 5 coded instead of years >= 5), which assigns the employee to the lower band. The test suite never contained the boundary values, or used mid-band values such as 3, 7, 12 and 20 only.

Assumed rules: 0 to 4 years: 5% of salary; 5 to 9: 10%; 10 to 14: 15%; 15 or more: 20%; performance rating 1 to 5 (multiplier applied per rating).

2. ECP

Dimension	Class	Representative
Years	Invalid: negative	-1
Years	Band A: 0 to 4	2
Years	Band B: 5 to 9	7
Years	Band C: 10 to 14	12
Years	Band D: 15 or more	20
Rating	Invalid: < 1 or > 5; non-numeric	0, 6, 'A'
Rating	Valid: 1, 2, 3, 4, 5 (one class per rating multiplier)	1, 3, 5

3. BVA test suite

Years	Expected band	Purpose
-1, 0, 1	Invalid, A, A	Lower end
4, **5**, 6	A, **B**, B	Boundary at 5
9, **10**, 11	B, **C**, C	Boundary at 10
14, **15**, 16	C, **D**, D	Boundary at 15
Very large (e.g. 60)	D	Upper robustness

Each boundary year is paired with the lowest, middle and highest rating (1, 3, 5), and each rating boundary (0, 1, 2, 4, 5, 6) is paired with one mid-band year. Expected payouts are computed independently from the rule table.

4. Why BVA/ECP are superior to exploratory testing here

- **Precision:** the logic is deterministic and numeric, with known cut-off points. Boundary tests find the exact fault; exploratory testing may not touch 5, 10 and 15 at all.
- **Repeatability and audit:** payroll is a regulated, financial function. Scripted tests give evidence that every band was verified, and rerun on every change.
- **Efficiency:** about 18 tests give complete band coverage. Exploratory sessions have no completeness measure.
- **Traceability:** every test maps to a rule in the compensation policy.
- Exploratory testing remains a useful complement for usability and unexpected scenarios, but it cannot replace boundary verification for financial calculations.

Question 7: Laboratory Temperature Control: Missed Alarm

Scenario: Complex if-else logic for heaters and coolers; the alarm did not trigger during a sensor-malfunction sequence.

1. Module logic (assumed) and CFG

Node	Logic
N1	Read T (temperature), sensorStatus, faultCount
N2	D1: if (sensorStatus == FAULT)
N3	D2: if (lastValidTemp > SAFE_MAX OR faultCount ≥ 3) [conditions A, B]
N4	Raise ALARM
N5	D3: else if (T < LOW)
N6	Heater ON
N7	D4: else if (T > HIGH)
N8	Cooler ON
N9	Idle (no action)
N10	D5: if (T > CRITICAL)
N11	Raise ALARM
N12	Exit

Flow: N1→N2. D1 true→N3; D1 false→N5. N3 true→N4, N3 false→N10. N5 true→N6, false→N7. N7 true→N8, false→N9. N4, N6, N8, N9 → N10. N10 true→N11→N12; false→N12.

Cyclomatic complexity: decision nodes D1 to D5 = 5, so V(G) = **6**. Counting each atomic condition separately (A and B), the condition-level complexity is 7.

2. Gaps in branch coverage

- Branch coverage only needs D2 to be true once and false once. A test where A is true (last valid temperature high) is enough for the 'true' branch, so condition B (repeated sensor faults) is never evaluated independently.
- The failing production sequence (repeated faults while the last valid temperature looked normal) exercises B alone. A branch-complete suite can pass without ever running it.
- Branch coverage also does not show that A and B each influence the outcome, so a masked or wrongly coded condition goes unnoticed.

3. White-box strategy for 100% MC/DC

MC/DC requires each condition to be shown, by a pair of tests that differ only in that condition, to change the decision outcome. For D2 = A OR B:

Test	A (last valid temp high)	B (faults ≥ 3)	D2 result	Shows independence of
M1	T	F	T (alarm)	A (with M3)
M2	F	T	T (alarm)	B (with M3)
M3	F	F	F (no alarm from D2)	baseline

Add the remaining decisions: D1 (fault / no fault), D3 (T < LOW true/false), D4 (T > HIGH true/false), D5 (T > CRITICAL true/false). Single-condition decisions need one true and one false test, and many of these combine with the D2 tests. Boundary values for the thresholds (LOW−0.1, LOW, HIGH, HIGH+0.1,

CRITICAL, CRITICAL+0.1) and the fault counter (2, 3, 4) are added, since MC/DC does not test numeric boundaries by itself.

The sensor-malfunction regression sequence becomes an explicit test: FAULT three times in a row with normal temperature, expecting the alarm on the third fault.

4. Impact on safety-critical reliability

- MC/DC (required by standards such as DO-178C for the most critical software) verifies each condition's real effect on the decision, which catches the masked-condition defect that escaped here.
- It needs only n + 1 tests per decision instead of 2^n, so it is thorough but still practical.
- A missed alarm in a laboratory is a safety event; MC/DC removes the class of logic faults most likely to cause it.

Question 8: Travel Portal: Negative Total with Multiple Promo Codes

Scenario: Certain promo combinations produce a negative Total Amount due.

1. Limitations of testing inputs in isolation

- Price, promo and tier are each valid individually; the defect appears only when their effects are combined (for example a flat ₹500 code on a ₹400 booking).
- Isolated tests never make the discount larger than the price, so the missing clamp or validation is not exercised.
- Testing every combination blindly is wasteful; a structured map of combinations is needed.

2. Decision table (assumed rules)

Assumed: percentage code = 10%; flat code = ₹500; Gold tier = extra 5%; total must never be below ₹0.

Rule	C1 Percent code	C2 Flat code	C3 Gold tier	C4 Discounts > price	Expected action
R1	N	N	N	-	Full price
R2	N	N	Y	-	Gold discount only
R3	Y	N	-	N	Percent (plus Gold if C3)
R4	N	Y	-	N	Flat (plus Gold if C3)
R5	Y	Y	-	N	Stack in defined order: percent, flat, Gold
R6	Y	N	-	Y	Total clamped to ₹0 (never negative)
R7	N	Y	-	Y	Total clamped to ₹0
R8	Y	Y	-	Y	Total clamped to ₹0 or second code rejected

'-' means don't-care; collapsing don't-cares reduces the 16 raw combinations of four conditions to 8 rules.

3. Optimized test set

Test	Price	Promo	Tier	Expected
TC1	2,000	None	Standard	2,000 (R1)
TC2	2,000	None	Gold	1,900 (R2)
TC3	2,000	10% code	Standard	1,800 (R3)
TC4	2,000	₹500 flat	Standard	1,500 (R4)
TC5	2,000	10% + ₹500 flat	Standard	1,300 (R5)
TC6	400	₹500 flat	Standard	0, not negative (R7)

Test	Price	Promo	Tier	Expected
TC7	400	10% + ₹500 flat	Gold	0, not negative (R8)
TC8	500	₹500 flat (boundary: exactly equal)	Standard	0
TC9	501 / 499	₹500 flat (just above/below)	Standard	1 / 0 (clamped)
TC10	2,000	Same code applied twice	Standard	Second use rejected
TC11	2,000	Expired / invalid code	Standard	Rejected, price unchanged
TC12	0	Any code	Any	Total 0, no negative

4. Why this maximizes detection with minimum redundancy

Each decision-table rule is tested once (no repeated logic), and extra tests are added only for the high-risk boundary where discount equals, just exceeds, or just falls short of the price. Twelve tests replace the dozens of full-factorial combinations while still exposing the negative-total defect, stacking-order errors, and invalid-code handling. A cause-effect graph over the same causes (codes, tier, discount > price) yields the same table and helps detect impossible combinations early.

Question 9: Database Migration Tool: Infinite Loops Despite High Statement Coverage

Scenario: Deeply nested validation loops; high statement coverage, but infinite-loop crashes in production.

1. Why statement coverage was insufficient

Statement coverage only requires that each statement runs once. A statement that is not written (an implicit else branch) can never be covered. Consider:

Node	Code
1	i = 0
2	while (i < N) [P1]
3	ok = validate(rec[i])
4	while (!ok) [P2]
5	if (repair(rec[i])) [P3]
6	ok = true
7	i++
8	end

One test where repair succeeds executes every statement (100% statement coverage). The case where repair() fails (the false edge of P3) is never run; ok stays false and the inner loop never terminates. The coverage report looked excellent while the dangerous edge was untested.

2. Cyclomatic Complexity

Predicate nodes P1, P2, P3 → **V(G) = 3 + 1 = 4** (verification: E − N + 2 gives the same result). So 4 independent paths are needed at minimum.

3. Path testing strategy

Path	Behavior	Test data
IP1	Outer loop never entered	N = 0
IP2	Record valid first time (inner loop skipped)	N = 1, valid record
IP3	Inner loop entered, repair succeeds	N = 1, repairable record

Path	Behavior	Test data
IP4	Inner loop entered, repair fails and the loop repeats	N = 1, unrepairable record (expect bounded retries and an error, not a hang)

Loop boundary tests: outer loop 0, 1, 2, N−1, N iterations; inner loop 0, 1, 2, MAX_RETRY−1, MAX_RETRY, MAX_RETRY+1 iterations. Run unrepairable-record tests under a timeout so non-termination fails the test instead of hanging the pipeline.

4. Justification with cyclomatic complexity

- V(G) = 4 gives the minimum number of tests (4) for basis path coverage, which guarantees branch coverage (and therefore the missing false edge of P3).
- A complexity this high, together with unbounded loops, signals risk. Recommended code fix: add a retry counter or timeout to the inner loop (while (!ok && retries < MAX)).
- Expected outcome: 100% branch and basis-path coverage, all loops proven to terminate, and zero infinite-loop crashes in regression.

Question 10: Food Delivery: Tracker Stuck After Driver Cancellation

Scenario: A state-based tracker (Ordered, Preparing, Out for Delivery, Delivered) freezes when a driver cancels mid-delivery.

1. Evaluation of the current black-box strategy

- The tracker was likely tested by checking single inputs and outputs. The failing behavior depends on the sequence of events and the current state, which ECP/BVA do not model.
- The model itself has a gap: there is no state for 'driver cancelled / reassigning', so the system has no valid transition out of Out for Delivery on a driver-cancel event.

2. State model

States: **Ordered, Preparing, Out for Delivery, Reassigning Driver, Delivered, Cancelled**. Events: restaurant accepts, food picked up, delivery confirmed, customer cancels, driver cancels, new driver assigned, reassignment timeout.

Current state	Event	Next state	Valid?
Ordered	Restaurant accepts	Preparing	Valid
Ordered	Customer cancels	Cancelled	Valid
Preparing	Food picked up	Out for Delivery	Valid
Preparing	Customer cancels	Cancelled (refund rules apply)	Valid
Out for Delivery	Delivery confirmed	Delivered	Valid
Out for Delivery	**Driver cancels**	**Reassigning Driver**	Valid (previously missing)
Reassigning Driver	New driver assigned	Out for Delivery	Valid
Reassigning Driver	Timeout (no driver)	Cancelled + refund / notify	Valid
Delivered	Any cancel event	Delivered (ignored)	Invalid transition
Ordered	Delivery confirmed	No change	Invalid transition

3. Redesigned test scenarios

- **0-switch (each transition once):** all valid transitions above, so every arrow is covered.

- **Mid-delivery cancel path:** Ordered → Preparing → Out for Delivery → driver cancels → Reassigning → new driver → Delivered.
- **Cancel with no replacement:** … driver cancels → Reassigning → timeout → Cancelled, with refund and notification, and the UI must leave the stuck state.
- **Repeated cancels (1-switch):** driver cancels twice in a row, then a third driver delivers.
- **Invalid events:** delivery confirmation while Preparing, cancel after Delivered, duplicate events; the state must not change and no crash may occur.
- **Non-functional state behavior:** app killed and reopened, or network lost, during Reassigning; the tracker must restore the correct state.

4. Why state transition testing succeeds where ECP/BVA fail

ECP and BVA treat each input as independent, so 'driver cancels' is simply one valid input value. Its correct outcome, however, depends on the state it arrives in, and the same event is valid in Out for Delivery but meaningless in Delivered. State transition testing makes history and sequence explicit, exposes missing states and missing transitions (the root cause here) and systematically tests invalid transitions. It therefore finds logic gaps in dynamic workflows that input-value techniques cannot see.

Question 11: Authentication Module: Brute-Force Weaknesses

Scenario: Multi-factor logic (password strength, OTP timing, IP location) still shows brute-force vulnerabilities.

1. Critical assessment of the current testing

- Black-box tests probably verified normal login success/failure but not repeated or abusive use (attempt counting, lockout, throttling).
- White-box coverage may have reached each branch once, but not the paths that matter for abuse, such as a failure counter that resets at the OTP stage or is keyed to a session or IP that the attacker can change.
- The two approaches were not linked: no one converted knowledge of the code paths into attack-style test inputs.

2. Formal path analysis (CFG)

Decision	Logic
D1	Password valid and meets strength policy?
D2	Failed-attempt counter ≥ limit (lock account)?
D3	OTP valid?
D4	OTP within time window and unused?
D5	IP location trusted / consistent?

Five decisions → **V(G) = 6** independent paths: (1) bad password, counter below limit; (2) bad password reaching lockout; (3) good password, wrong OTP; (4) good password, expired or reused OTP; (5) good password and OTP, untrusted IP (step-up or deny); (6) full success. Path analysis checks, for each path, whether the failure counter is incremented, persisted and enforced, and whether any path skips a check.

3. Hybrid strategy: Error Guessing + Path Analysis

Technique	Test ideas (defensive verification)	Defect targeted
Error guessing	Many wrong passwords from one IP; same account from many IPs; attempts split across new sessions	Counter keyed to wrong identifier; lockout bypass

Technique	Test ideas (defensive verification)	Defect targeted
Error guessing	OTP at 29 s, 30 s, 31 s of a 30 s window; reuse of a used OTP; many OTP guesses	OTP timing boundary; replay; no OTP attempt limit
Error guessing	Failure counter after a correct password followed by wrong OTP; counter reset behavior	Counter reset on partial success
Error guessing	Changed IP header values; IP change between password and OTP step	Weak IP check / trust of client headers
Path analysis	One test per basis path (6) with assertions on counter, lockout, logging and response time	Unchecked or skipped paths
Path analysis + BVA	Attempt limit −1, limit, limit + 1; lockout expiry boundary	Off-by-one in lockout
Non-functional	Uniform error messages and response times for known/unknown accounts	Username enumeration

4. Justification: enhanced security posture

- Path analysis guarantees every logical route through the authentication flow is verified; error guessing supplies the adversarial inputs that requirement-based tests never contain.
- Together they test both what the design intends (paths) and how an attacker would try to bend it (guessed misuse).
- Results feed back into the checklist: each new vulnerability becomes both a new guessed-error test and, if needed, a new path assertion.
- Recommended complements: rate limiting, progressive delays, account-level lockout independent of IP/session, and logging of failed attempts.

Question 12: E-Learning Platform: Upgrade on the Day of Expiry

Scenario: Tiered access (Free, Basic, Premium) by subscription date and course type; errors on upgrade exactly on the expiry day.

1. Adequacy of current criteria

The test criteria evidently covered 'active' and 'expired' subscriptions, but not the expiry date itself. The exact-day case is a boundary that is ambiguous in the requirement (is access valid through the whole expiry day or until its start?) and sits between two date classes, so it needs its own test and a documented rule.

2. ECP: date ranges (relative to the expiry date E)

Class	Date condition	Type	Representative
D1	Before subscription start	Invalid / no access	Start − 1 day
D2	Start ≤ date < E (active)	Valid	E − 10 days; E − 1 day
D3	Date = E (expiry day)	Boundary	E at 00:00:00, 12:00, 23:59:59
D4	Date > E (expired)	Invalid	E + 1 day

Tier classes: Free, Basic, Premium. **Course-type classes:** Free-level course, Basic-level course, Premium-level course. **Upgrade-path classes:** Free→Basic, Free→Premium, Basic→Premium, Basic→Basic (renewal), Premium→Basic (downgrade).

3. Structured test design

Test	Current tier / date class	Action	Expected result
U1	Basic / D2	Upgrade to Premium	Premium immediately; remaining Basic time handled per policy
U2	Basic / D3 (expiry day)	Upgrade to Premium	Premium active; no loss of access, no gap
U3	Basic / D3, 23:59:59	Upgrade completes after midnight	Premium access continues; date/time-zone logic consistent
U4	Basic / D4	Upgrade to Premium	New Premium period starts from the upgrade date
U5	Free / any	Upgrade to Basic / Premium	Access matches new tier immediately
U6	Premium / D3	Downgrade to Basic	Premium access until policy-defined cutoff
U7	Basic / D3	Renew same tier	Expiry extended; no double-charging
U8	Each tier × each course type	Open course	Allowed/denied per access matrix (3 × 3 = 9 tests)

4. Justification of completeness

- Every date class, tier and upgrade path is represented, and the exact expiry day and the time-of-day edges are tested separately.
- The 3 tiers × 3 course types access matrix is verified in full (9 combinations).
- Time-zone and midnight tests remove the usual root cause of expiry-day defects.
- The requirement ambiguity is resolved in writing before tests are finalised, so expected results are unambiguous.

Question 13: Smart-Home Lighting Controller: Switch-Case and Boolean Logic

Scenario: Switch-case statements and complex Boolean expressions in the dimming algorithm.

1. Risk of relying only on statement coverage

- A switch with a default or fall-through can reach every statement while skipping combinations of cases.
- A compound condition such as (lux < 30 AND motion) OR (nightMode AND NOT override) can be 'true' via one operand alone. Statement coverage never shows that the other operands were evaluated or even correctly written.
- A missing else/default branch contains no statement, so it is invisible to statement coverage but can leave a dimmer value stuck or undefined.
- In hardware-software integration, a wrong boolean decision means the wrong actuator state, which can be hard to diagnose later.

2. Branch coverage: switch-case

Mode value	Test	Expected output
AUTO	lux sensor 10, 50	Dim up or down per algorithm
MANUAL	user level 40	Fixed manual level
NIGHT	any lux	Night level
default (unknown / corrupt value)	mode = 99	Safe default state; error logged
Case with missing break	Sequence AUTO then NIGHT	No fall-through side effect

3. Condition coverage / MC/DC on the Boolean expression

Decision: (A AND B) OR (C AND NOT D), where A = lux < 30, B = motion detected, C = night mode, D = manual override.

Test	A	B	C	D	Decision	Pair showing independence
T1	T	T	F	F	T	A (with T2), B (with T3)
T2	F	T	F	F	F	A
T3	T	F	F	F	F	B
T4	F	T	T	F	T	C (with T2)
T5	F	T	T	T	F	D (with T4)

These 5 tests (n + 1 for n = 4 conditions) give 100% decision, condition and MC/DC coverage for this expression, compared with 16 for exhaustive testing. Add boundary values for the sensor threshold (lux 29, 30, 31) and dimming limits (0% and 100%).

4. Improved white-box strategy

- Branch coverage for all switch cases including default.
- MC/DC for every compound Boolean expression.
- Boundary tests on sensor thresholds, plus simulated sensor failure values (0, maximum, out-of-range, stale).
- Hardware-in-the-loop or stubbed sensor/actuator tests that replay the same vectors against the real interface.

5. Justification

Branch and condition coverage expose the untested decision outcomes that statement coverage hides, and MC/DC confirms every sensor input genuinely influences the output. Because the same vectors are replayed at the hardware interface, mismatches between software logic and device behavior are found in the lab rather than in a customer's home, lowering integration risk.

Question 14: Logistics: Zero and Negative Weight Not Caught

Scenario: Shipping cost depends on weight, destination zone and delivery speed; errors with zero or negative weights were missed.

---

1. Why the input-validation testing failed

- Tests used only valid weights, so the validation rules for zero, negative and non-numeric input were never exercised.
- Standard BVA checks only the minimum and maximum of the valid range and skips the first invalid values on each side.
- Without validation, a negative weight can yield a negative or zero shipping cost, which becomes revenue loss or fraud.

Assumed valid weight: 0.1 kg to 50 kg; zones Z1 to Z4; speeds Standard / Express / Overnight.

2. Robustness BVA for weight

Test	Weight (kg)	Class	Expected result
W1	-5 / -0.01	Invalid (negative)	Rejected with clear error
W2	0	Invalid (zero)	Rejected
W3	0.09	Just below minimum	Rejected

Test	Weight (kg)	Class	Expected result
W4	0.1	Minimum	Accepted, cost computed
W5	0.11	Just above minimum	Accepted
W6	49.9	Just below maximum	Accepted
W7	50	Maximum	Accepted
W8	50.01	Just above maximum	Rejected or freight-rate rule per policy
W9	empty, abc, NaN, 1e309, 2.555 (precision)	Invalid format	Rejected, no crash

Zone and speed receive the same treatment: valid values (Z1 to Z4; three speeds), plus invalid values (Z0, Z5, empty, unknown speed). Representative combinations of weight × zone × speed use valid weights at the boundaries (0.1, 50) for each zone and speed.

3. Necessity of negative scenarios

- **Financial integrity:** invalid weights must never produce zero or negative charges.
- **Robustness:** real users, API clients and integrations send bad data; the system must fail safely with a clear message, not crash or corrupt state.
- **Security:** unvalidated input is a common attack surface (overflow, injection through numeric fields).
- **Defect yield:** the highest-defect areas are the invalid boundaries; skipping them leaves the majority of input-handling faults undetected.
- **Cost:** an error caught in test is far cheaper than a mispriced shipment or customer dispute in production.

Question 15: Medical Imaging: Prime-Sized Data Failures in Parallel Loops

Scenario: Parallel processing loops give inconsistent results only when the dataset size is a prime number.

1. Why random testing is inadequate

- Primes are sparse (density about 1/ln N), so random sizes rarely produce a prime. Each random run also gives no reason to expect the prime case matters.
- The cause is structural: data is split across T workers using N / T; when N is prime (and larger than T), N % T is never 0, so the remainder chunk is always non-empty. If the remainder is dropped or processed twice, results are wrong only for such sizes.
- Random testing has no coverage target, no oracle and no repeatability, which is unsuitable for a medical, mathematical module.

2. Module structure and Basis Path Testing

Node	Logic
1	Read N; if (N ≤ 0) return error [P1]
2	chunk = N / T; rem = N % T
3	for t = 0 to T−1 [P2]
4	start = t × chunk; end = start + chunk
5	if (t == T−1 AND rem ≠ 0) end = N [P3]
6	worker loop over start..end [P4]
7	merge results; check total count = N [P5: consistency check]

Five predicate nodes, so **V(G) = 6** basis paths: N ≤ 0; empty worker range (N < T); remainder = 0; remainder ≠ 0 (remainder handling); single-element worker loop (N = T); consistency check fails. Each path gets test data:

Path	Test data	Why
B1	N = 0 and N = −1	Input guard
B2	N = 1 (N < T)	Empty chunks; chunk = 0
B3	N = 1024 (divisible by T)	Remainder = 0 path
B4	N = 97, 7919 (primes)	Remainder ≠ 0 path; the failing case
B5	N = T and N = T + 1	Edge between divisible and remainder cases
B6	Corrupt/forced mismatch	Consistency check triggers error, not silent wrong output

Also include sizes of the form k·T − 1, k·T, k·T + 1 and a large prime near the memory limit.

3. Ensuring accuracy and stability

- **Oracle:** compare every parallel result against a trusted serial implementation (exact for integer work, within a stated tolerance for floating point).
- **Invariants:** the sum of chunk sizes equals N; each element is processed exactly once.
- **Repetition and thread counts:** run each case several times with T = 1, 2, 4, max cores to expose races that path testing cannot see; add a thread sanitizer.
- **Property-based test, targeted:** generate sizes from primes and from composites as separate partitions, with a fixed seed, so random testing becomes systematic.

4. Justification

Basis Path Testing guarantees that both the divisible and non-divisible partition paths are executed by design, instead of by luck, and the prime-size inputs are chosen because they always force the remainder path. The serial oracle and invariants prove mathematical accuracy, and the repeated multi-thread runs support system stability. This produces repeatable evidence of correctness for a safety-relevant imaging system.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification